

Capítol 3: Models i camps bàsics

Al final del [capítol anterior](#), vam ser capaços de crear un mòdul d'Odoo. No obstant això, de moment encara és una carcassa buida que no ens permet emmagatzemar cap dada. En el nostre mòdul immobiliari, volem emmagatzemar la informació relacionada amb les propietats (nom, descripció, preu, superfície habitable...) en una base de dades. El *framework* d'Odoo proporciona eines per a facilitar les interaccions amb la base de dades.

Abans d'avançar en l'exercici, assegureu-vos que el mòdul `estate` està instal·lat, és a dir, ha d'aparèixer com a 'Instal·lat' a la llista d'Aplicacions.



No utilitzeu variables globals mutables

Una única instància d'Odoo pot executar diverses bases de dades en paral·lel dins del mateix procés de Python. En cadascuna d'aquestes bases de dades poden estar instal·lats mòduls diferents, per tant, no podem dependre de variables globals que s'actualitzarien depenent dels mòduls instal·lats.

Mapeig objecte-relacional (ORM)



Objectiu

Al final d'aquesta secció, s'hauria d'haver creat la taula `estate_property`:

```
1  ``text
2  $ psql -d rd-demo
3  rd-demo=# SELECT COUNT(*) FROM estate_property;
4  count
5  -----
6  0
7  (1 row)
8  ``
```

Un component clau d'Odoo és la capa [ORM](#). Aquesta capa evita haver d'escriure manualment la major part de l'[SQL](#) i proporciona serveis d'extensibilitat i seguretat.

Els objectes de negoci es declaren com a classes de Python que estenen `odoo.models.Model`, la qual cosa els integra en el sistema automàtic de persistència.

Els models es poden configurar establint atributs en la seua definició. L'atribut més important és `_name`, que és obligatori i defineix el nom del model en el sistema Odoo. Ací teniu una definició mínima d'un model:

Definició del paràmetre `_name` d'un model

```
1 from odoo import models
2
3 class TestModel(models.Model):
4     _name = "test_model"
```

Aquesta definició és suficient perquè l'ORM genere una taula a la base de dades anomenada `test_model`. Per convenció, tots els models es troben en un directori `models` i cada model es defineix en el seu propi fitxer Python.

Feu una ullada a com està definida la taula `crm_recurring_plan` i com s'importa el fitxer Python corresponent:

1. El model es defineix al fitxer `crm/models/crm_recurring_plan.py` (vegeu-ho [ací](#)).
2. El fitxer `crm_recurring_plan.py` s'importa a `crm/models/__init__.py` (vegeu-ho [ací](#)).
3. La carpeta `models` s'importa a `crm/__init__.py` (vegeu-ho [ací](#)).

Exercici: Definiu el model de propietats immobiliàries

Basant-vos en l'exemple donat en el mòdul CRM, creeu els fitxers i la carpeta adequats per a la taula `estate_property`.

Quan els fitxers estiguen creats, afegiu una definició mínima per al model `estate.property`.

Qualsevol modificació dels fitxers Python requereix reiniciar el servidor d'Odoo. Quan reiniciem el servidor, afegirem els paràmetres `-d` i `-u`:

```
1 $ ./odoo-bin --addons-path=addons,..../enterprise,..../tutorials/ -d rd-demo
  -u estate
```

`-u estate` significa que volem actualitzar el mòdul `estate`, és a dir, l'ORM aplicarà els canvis a l'esquema de la base de dades. En aquest cas, crearà una nova taula. `-d rd-demo` significa que l'actualització s'hauria de realitzar a la base de dades `rd-demo`. L'opció `-u` s'hauria d'utilitzar sempre en combinació amb `-d`.

Durant l'inici, hauríeu de veure els següents avisos:

```
1 ...
2 WARNING rd-demo odoo.models: The model estate.property has no _description
3 ...
  WARNING rd-demo odoo.modules.loading: The model estate.property has no
```

```
4 | access rules, consider adding one...
5 | ...
```

Si és així, aneu per bon camí! Per assegurar-vos-en, torneu-ho a comprovar amb `psql` com s'ha demostrat a l'**Objectiu**.



Exercici: Afegiu una descripció

Afegiu un `_description` al vostre model per a desfer-vos d'un dels avisos.

Camps del model

Els camps s'utilitzen per a definir què pot emmagatzemar el model i on s'emmagatzema. Els camps es defineixen com a atributs a la classe del model:

Exemple de camps a un model

```
1 | from odoo import fields, models
2 |
3 | class TestModel(models.Model):
4 |     _name = "test_model"
5 |     _description = "Test Model"
6 |
7 |     name = fields.Char()
```



El camp `name` és del tipus `Char`, que es representarà com a un `str` Unicode en Python i un `VARCHAR` en SQL.

Tipus

Objectiu

Al final d'aquesta secció, s'haurien d'haver afegit diversos camps bàsics a la taula

estate_property:

```
1  $ psql -d rd-demo
2
3  rd-demo=# \d estate_property;
4
5  "public.estate_property"
6  Column          | Type          | Collation | Nullable
7  |                  | Default      |           |
8  -----+-----+-----+-----+-----
9  id                | integer      |           | not
10 null | nextval('estate_property_id_seq'::regclass)
11 create_uid        | integer      |           |
12 |
13 create_date       | timestamp without time zone |           |
14 |
15 write_uid         | integer      |           |
16 |
17 write_date        | timestamp without time zone |           |
18 |
19 name              | character varying |           |
20 |
21 description       | text         |           |
22 |
23 postcode          | character varying |           |
24 |
25 date_availability | date         |           |
26 |
27 expected_price    | double precision |           |
28 |
29 selling_price     | double precision |           |
30 |
31 bedrooms          | integer      |           |
32 |
33 living_area       | integer      |           |
34 |
35 facades           | integer      |           |
36 |
37 garage            | boolean      |           |
38 |
39 garden            | boolean      |           |
40 |
41 garden_area       | integer      |           |
42 |
43 garden_orientation | character varying |           |
44 |
45 Indexes:
46     "estate_property_pkey" PRIMARY KEY, btree (id)
47 Foreign-key constraints:
48     "estate_property_create_uid_fkey" FOREIGN KEY (create_uid)
49 REFERENCES res_users(id) ON DELETE SET NULL
50     "estate_property_write_uid_fkey" FOREIGN KEY (write_uid)
51 REFERENCES res_users(id) ON DELETE SET NULL
```

Hi ha dues grans categories de camps: els **camps 'simples'**, que són valors atòmics emmagatzemats directament a la taula del model, i els **camps 'relacionals'**, que enllacen registres (del mateix o de diferents models).

Alguns exemples de camps simples són `Boolean`, `Float`, `Char`, `Text`, `Date` i `Selection`.



Exercici: Afegiu camps bàsics a la taula Propietat Immobiliària

Afegiu els següents camps bàsics a la taula:

Camp	Tipus
name	Char
description	Text
postcode	Char
date_availability	Date
expected_price	Float
selling_price	Float
bedrooms	Integer
living_area	Integer
facades	Integer
garage	Boolean
garden	Boolean
garden_area	Integer
garden_orientation	Selection

El camp `garden_orientation` ha de tindre 4 valors possibles: 'North' (Nord), 'South' (Sud), 'East' (Est) i 'West' (Oest). La llista de selecció es defineix com a una llista de tuples, vegeu-ne un exemple [ací](#).

Quan els camps s'hagen afegit al model, reinicieu el servidor amb `-u estate`.

```
1 | $ ./odoo-bin --addons-path=addons,..../enterprise,..../tutorials/ -d rd-demo
    -u estate
```

Connecteu-vos a `psql` i comproveu l'estructura de la taula `estate_property`. Notareu que també s'han afegit un parell de camps addicionals a la taula. Els revisarem més endavant.

Atributs comuns

Objectiu

Al final d'aquesta secció, les columnes `name` i `expected_price` no haurien de poder ser nul·les (not nullable) a la taula `estate_property`:

```
1 | rd-demo=# \d estate_property;
2 |
3 |                               Table
4 | "public.estate_property"
5 |      Column      |      Type      | Collation | Nullable
6 | -----+-----+-----+-----+
7 | ...
8 | name              | character varying |           | not
9 | null |
10 | ...
11 | expected_price    | double precision  |           | not
12 | null |
13 | ...
```

De manera semblant al propi model, els camps es poden configurar passant-los atributs de configuració com a paràmetres:

```
1 | name = fields.Char(required=True)
```

Alguns atributs estan disponibles en tots els camps, ací teniu els més comuns:

- `string` (`str`, per defecte: el nom del camp): L'etiqueta del camp a la interfície d'usuari (visible pels usuaris).
- `required` (`bool`, per defecte: `False`): Si és `True`, el camp no pot estar buit. Ha de tindre un valor per defecte o bé se li ha d'assignar un valor sempre que es cree un registre.
- `help` (`str`, per defecte: `''`): Proporciona un text d'ajuda llarg emergent per als usuaris a la interfície d'usuari.
- `index` (`bool`, per defecte: `False`): Sol·licita que Odoo cree un [índex de base de dades](#) a la columna.



Exercici: Establiu atributs per a camps existents

Afegiu els atributs següents:

Camp	Atribut
name	required
expected_price	required

Després de reiniciar el servidor, cap dels dos camps hauria de poder ser nul.

Camps automàtics



Referència

La documentació relacionada amb aquest tema es pot trobar a [camps automàtics](#).

És possible que hàgeu notat que el vostre model té alguns camps que mai heu definit. Odoo crea uns quants camps en tots els models. Aquests camps estan gestionats pel sistema i no s'hi pot escriure, però es poden llegir si és útil o necessari:

- `id` (`Id`): L'identificador únic per a un registre del model.
- `create_date` (`Datetime`): Data de creació del registre.
- `create_uid` (`Many2one`): Usuari que va crear el registre.
- `write_date` (`Datetime`): Data de l'última modificació del registre.
- `write_uid` (`Many2one`): Usuari que va modificar per última vegada el registre.

Ara que hem creat el nostre primer model, anem a [afegir un poc de seguretat!](#)